Pergamon

0305-0548(94)00035-2

*Computers Ops Res.* Vol. 22, No. 5, pp. 465–478, 1995
Elsevier Science Ltd. Printed in Great Britain

# A CAPACITATED GENERAL ROUTING PROBLEM ON MIXED NETWORKS

Ram Pandit† and B. Muralidharan‡

Department of Industrial and Manufacturing Systems Engineering, Iowa State University, Ames, IO 50011, U.S.A.

*(Received January 1992; in revised form May 1994)*

**Scope and Purpose**—The problem of routing vehicles arises very frequently in practice. The difficulty of the problem and its solution approaches are very sensitive to the application and the characteristics of the travel network. In this paper, we present a solution approach for the problem of routing a single vehicle over a mixed and disconnected network. A number of real life vehicle routing applications in which travel must be made on some specified streets belong to this class of problem. Some examples of such problems are: waste collection from residential areas, street sweeping and rural postman problem. The designed solution procedures are fast and are implemented on a personal computer.

**Abstract**—This study addresses the problem of routing vehicles over the specified segments of a mixed network. The routing problem is modeled as a Capacitated General Routing Problem (CGRP). The solution to the problem is a set of routes for the vehicles which are consistent with the capacity and work-hours constraints. For the CGRP, we present a heuristic procedure to solve for large size problems. Both route-first-partition-next (RFPN) and partition-first-and-route-next (PFRN) approaches are discussed. Computational results of the developed procedure are also presented.

## 1. INTRODUCTION

This paper addresses the problem of routing vehicles over the specified segments of a mixed network. Such problems arise very frequently in real-life applications of waste collection [1], street sweeping [2], routing of sanitation vehicles [3], routing of electric meter readers [4], school bus routing [5] and other problems of this nature.

The routing problem in which travel is done over the streets of a network such that each street is covered at least once and the total distance traveled is minimized is known as the Chinese Postman Problem (CPP). The other arc-routing problems are variants of the CPP. For example, if there is a limit on the vehicle capacity or shift time then the resulting problem is called the Capacitated Chinese Postman Problem (CCPP). Sometimes it is required that only some specified street segments of the network need be visited. Then the routing problem is called the Rural Postman Problem (RPP). However, if routing is to be done on some specified street segments and some specified nodes then the problem is called the General Routing Problem (GRP). In the RPP and the GRP if the capacity and time restrictions are explicitly considered then the resulting problems are called the Capacitated RPP (CRPP) and the Capacitated GRP (CGRP), respectively. In the problem addressed in this study, the vehicles are required to travel on specified street segments and specified nodes. Therefore, we model this routing problem as a CGRP. Classification of different vehicle routine and scheduling problems and their solution strategies are discussed in Refs [6–9]. There are a number of software systems in practice today which address various applications of the capacitated routing problem. A survey and comparison of these software packages are given in Refs [10, 11].

†Ram Pandit is currently a Senior Systems Engineer at Consolidated Freightways, Inc. at Portand, OR. His interests are in solving large scale real-life discrete optimization problems. He has a Ph.D. in Operations Research and Manufacturing Systems from the University of Illinois at Urbana-Champaign. Prior to his current position, he has held academic positions at the Industrial Engineering Departments of Iowa State University and Georgia Institute of Technology.

‡B. Muralidharan is a graduate student at the Industrial and Manufacturing Systems Engineering at Iowa State University, where he is pursuing his Ph.D. in Industrial Engineering. His research interests are in Discrete Optimization and the Analysis of Manufacturing Systems. This paper is part of his Master's thesis.

The CPP and its variants are well studied in the literature. The solution procedures to CPP are sensitive to the nature of travel network. Edmonds and Johnson [12] give polynomial algorithms for CPP when the travel network is completely directed or undirected. However, the problem becomes difficult to solve (NP-complete) if the travel network contains both the directed and undirected arcs. Such a travel network is called a *mixed network*. Frederickson [13, 14] and Minieka [15] study the CPP on a mixed network and give approximate solution procedures for this problem. Since the CPP is a relaxed form of the CCPP (obtained by ignoring capacity and time restrictions), an optimal solution to the CPP forms a lower bound to the CCPP.

In practice, solutions to large CCPP are obtained using heuristic solution procedures. These procedures fall into two broad classes: (i) route-first-partition-next (RFPN) and (ii) partition-first-and-route-next (PFRN). The RFPN procedures first form a large CPP tour by ignoring the restrictions on the vehicle capacity and tour length and then partition this large tour into smaller vehicle routes consistent with the restrictions on vehicle capacity and route length. One limitation of this procedure is to control the regional overlapping of the routes which may be undesirable from an administrative point of view. To overcome this, the PFRN approach is used. The PFRN approach first partitions the large region into smaller zones and then routing is performed for each zone separately. This method reduces the interweaving of routes, however the overall solution value depends upon the partitioning of the region which is done subjectively.

Bodin *et al.* [1–3] solve the CCPP in which all arcs are directed using a slightly modified Edmonds and Johnson's algorithm to solve the CPP. They discuss both the RFPN and the PFRN approaches.

Procedures which build tours by successive augmentation of arcs are studied by Golden *et al.* [16, 17]. They compare the performance of two heuristics for the CCPP, namely, Golden and Wong's Augment-Merge heuristic and the Construct-Strike algorithm by Christofides with their Path Scanning algorithm for the CCPP on an undirected connected network. The Augment-Merge and Path Scanning algorithms perform better than Construct-Strike algorithm when compared with the lower bound. The Augment-Merge takes more computation time when compared with the other two algorithms.

Wen-Lea Pearn [18] compares various algorithms for solving the CCPP: Construct-Strike algorithm by Christofides [19], the Path-Scanning and Augment-Merge algorithms by Golden *et al.* [16], the Modified Construct-Strike and Random Path-Scanning algorithms by Pearn [20], and the Augment-Insert Algorithm by Pearn [18]. He shows that Augment-Insert performs better than other algorithms when compared with the lower bound. The computation time for the Augment-Insert algorithm is considerably less than the Modified Construct-Strike algorithm, Construct-Strike algorithm and Augment-Merge Algorithms.

The GRP is studied by Orloff [21, 22], and Lenstra and Rinnooy Kan [23]. However, these studies do not consider the restrictions on vehicle capacities and available time.

In the case of graphs with disconnected components the augmentatioon procedures work best when the arcs are directed. This is because there is a unique way to travel from one arc to another. In the case of a mixed network or when the arcs are undirected, these procedures are not easy. This is due to the large number of possible ways to combine arcs. Therefore, for mixed graphs with disconnected components other solution strategies need to be examined. In this paper we present a heuristic approach which develops routes for the vehicles when the set of edges to be traveled form a mixed and disconnected graph.

The remainder of the paper is organized as follows. The overall problem is formally described in Section 2. The solution approach developed is explained in detail in Section 3. Section 4 presents lower bounds and computational results. Section 5 concludes the paper with a summary. Descriptions of the procedures used in the solution algorithm are included in Appendix I.

## 2. PROBLEM DESCRIPTION

We consider the problem of routing vehicles over the streets of residential areas and to convenience stores located within these areas. The streets used for traveling define a travel network. In this network, street segments allowing two-way (one-way) travel form edges (arcs) and the points of intersection of street segments define nodes. Furthermore, locations of convenience stores are also

represented as nodes. The street segments and the nodes of the network from which collection is to be made are called *specified* edges (arcs) and *specified* nodes respectively. It is assumed that the vehicles start from a single depot and return there after every trip. It is possible that a vehicle may make multiple trips during a work day. This depot is also represented as a node in the network. In this network, since we need to visit both the specified edges (arcs) and the nodes, the problem is modeled as a CGRP. CGRP addresses a multi-vehicle routing problem. To solve a multi-vehicle routing problem, it is essential to first devise an effective solution procedure for a single vehicle case. This procedure can then be extended to the multi-vehicle case.

The CGRP may formally be described as follows. Let $G_0=(N_0, R_0)$ be a strongly connected graph describing the complete travel network for the vehicles, where $N_0$ is the set of nodes representing the intersection of the streets and locations of convenience stores, and $R_0$ is the set of street segments on which vehicles can travel. Let $A_0$, a subset of $R_0$, be the set of *specified* arcs representing one-way streets (directed arcs) and $E_0$ also a subset of $R_0$ be the set of *specified* edges representing the streets allowing bi-directional travel. The segments of streets in $A_0$ and $E_0$ must be visited by the vehicles. Similarly, let $V_0$, a subset of $N_0$, be the specified set of nodes which must be visited. Let node 1 represents a central depot with $K$ vehicles, with $W_k$ being the capacity of vehicle $k$. Let $C(i, j)$. $(i, j) \in \{A_0 \cup E_0\}$ be the length of the segment $(i, j)$ and let $q_{ij}>0$ be the expected amount of material that can be collected from it. There is a demand (weight) $p_i>0$ associated with each node $i \in V_0$, which represents the estimated amount of material to be picked at location $i$. Let $T$ be the available shift time during which the collection is to be made. Also, let $t_{ij}$ be the total pick up time required for street $(i, j)$ which depends upon the number of houses on street $(i, j)$ and $s_i$ be the pick up time required for node $i$. The vehicle is assumed to travel with an average constant speed. Thus, the travel time through a street segment $(i, j)$, while collection is being made from it, is given as $[C(i, j)/\text{Avg. speed}]+t_{ij}$; otherwise the travel time is equal to $C(i, j)/\text{Avg. speed}$.

The CGRP is now to find the number of cycles, each passing through the depot (node 1), which satisfy the demands of the nodes belonging to the set $V_0$ and the demands of the arcs belonging to the set $\{A_0 \cup E_0\}$ and minimizes the total distance traveled. Moreover, the capacity and time constraints in each case must be satisfied.

## 3. SOLUTION PROCEDURES

In our solution procedure to CGRP, we first condense the large travel network by considering only those arcs and nodes from which collection is to be made. The condensed network is generally a disconnected network with a number of small components. Since, for tours to be feasible, there should be a path between every two nodes, we add cheapest (minimum cost) edges and arcs to this condensed graph such that the condensed graph is strongly connected. To generate individual vehicle routes, we use a RFPN approach. First, a CPP tour is formed on the strongly connected graph. In this tour it is possible that some edges (arcs) are traveled more than once. If we explicitly augment the graph with the extra copies of these edges (arcs) equal to the number of times they are visited, the resulting graph becomes Eulerian. The Euler tour covers all the specified edges (arcs) and nodes. This Euler tour is then partitioned into individual vehicle routes.

As discussed earlier, this procedure when applied to the overall region (a city) may create interweaving routes which are difficult to manage administratively. However, in practice the overall region is generally partitioned into service zones (suburbs) and service is provided to each zone individually. Moreover, not all zones are serviced on the same day. The solution approach described below can be used for each zone individually; thereby reducing the regional overlapping of the routes and the management related problems.

### *3.1. Procedure to Solve CGRP*

Golden and Wong [24] show that even the 0.5 approximate CCPP which attempts to find a solution with a cost less than 1.5 times the value of the optimal solution is NP-hard. Since CCPP is a relaxation of the CGRP, the 0.5 approximate CGRP is also NP-hard. Therefore, approximate solution procedures are developed to solve the large-size real-life problems. The algorithm ROUTE1 solves CGRP on mixed networks and is explained below.

*ROUTE1 Algorithm*

The procedure starts with the condensed sub-graph of the travel network in which only those arcs, edges and nodes are considered which needs to be visited. Since the condensed sub-graph is generally disconnected, the procedure first augments the sub-graph with cheapest arcs and edges such that the subgraph becomes strongly connected. Finally, the strongly connected sub-graph is converted into an Eulerian graph which admits a tour. Following each step of the ROUTE1 algorithm is explained in detail. Descriptions of the algorithms used are given in Appendix I.

*Step 1.* The algorithm EXTRACT extracts the arcs, edges and nodes that need to be visited from the initial graph $G_0$ and forms a condensed graph, say $G_1=(V_1, R_1)$. The vertex set $V_1$ contains the set $V_0$ and the vertices associated with links in $\{A_0 \cup E_0\}$. The set $R_1=\{A_0 \cup E_0\}$. The condensed graph is generally a disconnected graph.

*Step 2.* The disconnected components of $G_1$ are joined together by finding a Minimum Spanning Tree (MST) between these components. To find a MST, we first form a complete graph $H=(W, E)$ from graph $G_1$, where $W$ is the set of vertices and $E$ is the edge set. The vertices of set $W$ are formed by contracting each disconnected component as a composite node. Thus, the number of vertices in $W$ are equal to the number of disconnected components in $G_1$. An edge $(u, v)$ in the set $E$ is a composite edge corresponding to the shortest path in $G_0$ connecting components $u$ and $v$ of graph $G_1$. It is possible that a shortest path in graph $G_0$ may contain both arcs and edges. If a shortest path contains arcs then it is represented as a composite arc oriented in the direction of the arcs on this path. If a shortest path contains only edges, it is represented as an composite edge. In edge set $E$ all composite arcs are initially considered as composite edges by temporarily ignoring their direction.

A MST is formed in graph $H$. Once the MST is formed, the direction of arcs are restored and graph $G_1$ is augmented with the links of MST. The composite nodes are expanded back to their original forms. Algorithm MINSPA finds the MST connecting the disconnected components as in Ref. [25]. Let $G_2=(V_2, R_2)$ be the connected graph after Step 2. The vertex set $V_2=V_1$ and $R_2=\{R_1$ and the composite links added due to MST$\}$.

After restoration of direction on arcs in the connected graph it is possible that some nodes have zero in-degree or zero out-degree. To ensure that such nodes are reachable from other nodes and vice-versa, we make the graph $G_2$ strongly connected. This is accomplished as follows.

*Step 3.* A strongly connected component (SCC) of $G_2$ is a subgraph of $G_2$ in which for every pair of vertices $u$ and $v$ there exists a path from $u$ to $v$ and a path from $v$ to $u$. Tarjan [26] gives an algorithm, which is polynomial in the number of nodes and arcs of the graph, to identify all its SCCs. However, the general problem of finding a SCC such that the component is of minimum weight remains NP-Complete. Eswaran and Tarjan [27] show that a special case of this problem in which all weights are equal is solvable in linear time with respect to the number of arcs.

To make $G_2$ a strongly connected graph, we need to add additional links to $G_2$ such that there is path between every pair of vertices in $G_2$. Moreover since the objective is to minimize the total distance traveled, this augmentation should be such that the sum of the lengths of these links is a minimum. The problem of selecting additional links is equivalent to the problem of finding minimum weight strongly connected components of a graph. We present a fast heuristic to solve this problem. This procedure is described as follows.

Consider graph $G_2$. Using Tarjan's algorithm we identify all SCCs of $G_2$. Let $k$ be the number of SCCs of $G_2$. Using graph $G_2$, we form a complete di-graph say, $\Omega=\{\Psi, \Gamma\}$, where $\Psi$ is the set of vertices and $\Gamma$ is the set of arcs. This construction is described below:

Each vertex in the set $\Psi$ is formed by contracting a SCC of $G_2$. Thus there are $k$ vertices in $\Psi$, each corresponding to a SCC of $G_2$. Let $\Gamma_1$ be the set of arcs in $G_2$ between these vertices thus formed. Note that there are no edges between the vertices of $\Psi$ in $G_2$. After contraction, it is possible that between a pair of nodes, say $u$ and $v$, there are multiple arcs from $u$ to $v$ and from $v$ to $u$. Therefore, between every pair (in each direction) we keep only one arc which has the minimum length and discard the others. We temporarily set the lengths of arcs $\Gamma_1$ equal to zero.

Now $\Omega$ is a connected graph in which each arc is of length zero. However, due to the orientations of the connecting arcs there may not be a path between every pair of vertices. To make $\Omega$ complete, we find shortest paths in $G_0$ between every pair of SCC of $G_2$. Again these shortest paths are represented as composite arcs and edges of length equal to the lengths of the corresponding shortest

paths. Since $\Omega$ is a di-graph we represent each composite edge by two directed composite arcs in $\Omega$. Let $\Gamma_2$ be the set of these composite arcs. Now the arc set $\Gamma = \Gamma_1 \cup \Gamma_2$. Thus $\Omega$ is a complete digraph with only two arcs between every pair of vertices. Let $C$ be a matrix of order $|\Psi| \times |\Psi|$ where an element $C_{ij}$ represents the length between vertices $i$ and $j$ in $\Omega$. We use $C$ to identify the required augmentation.

The heuristic procedure to identify the augmenting arcs is based on the Additive Bounding Procedure of Fischetti and Toth [28, 29] and is as follows.

Let node 1 be considered as the root node in $\Omega$. We solve a 1-Arborescence problem on $C$. Let $T_1$ be the resulting tree and $C'$ be the matrix of reduced costs. In $T_1$ there is a path from node 1 to every other node. Next a 1-Anti Arborescence problem is solved on $C'$. Let $T_2$ be the tree and in $T_2$ there is a path from every node to root node 1. Thus $T_1 \cup T_2$ forms a SCC of $\Omega$. The solution procedure for the Arborescence problem and computation of reduced costs are given in Fischetti and Toth [30]. To augment $G_2$, the arc set is identified as $\{T_1 \cup T_2\} \cap \Gamma_2$. The arcs corresponding to the composite edges are augmented as composite edges. Now $G_2$ is a strongly connected graph. This procedure is demonstrated with an example.

*Example.* Consider the cost matrix $C$ as shown in Fig. 1(a). The solution to the 1-Arborescence problem is shown in Fig. 1(b). The resulting tree is shown by continuous lines and the other arcs for which the reduced cost is zero are shown with "thick" dashed lines. The root node 1 has a path to every other node in the admissible graph. The 1-Anti Arborescence problem is solved on the resulting reduced cost matrix $C'$ and the solution to the 1-Anti Arborescence problem is shown in Fig. 1(c). The additional arcs added to the above tree by the 1-Anti Arborescence problem are

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 9999 | 38 | 28 | 26 | 24 | 30 | 16 | 10 | 49 | 36 |
| 33 | 9999 | 48 | 34 | 35 | 37 | 18 | 45 | 23 | 29 |
| 22 | 14 | 9999 | 41 | 44 | 13 | 12 | 11 | 10 | 26 |
| 34 | 45 | 27 | 9999 | 44 | 48 | 19 | 34 | 35 | 27 |
| 44 | 15 | 28 | 44 | 9999 | 34 | 28 | 29 | 27 | 15 |
| 14 | 34 | 10 | 10 | 38 | 9999 | 42 | 34 | 47 | 27 |
| 29 | 31 | 11 | 24 | 32 | 25 | 9999 | 34 | 26 | 48 |
| 17 | 41 | 19 | 24 | 19 | 45 | 11 | 9999 | 27 | 11 |
| 30 | 21 | 33 | 43 | 11 | 11 | 17 | 18 | 9999 | 44 |
| 11 | 36 | 11 | 22 | 35 | 17 | 45 | 20 | 12 | 9999 |

Fig. 1(a). Cost matrix for the example problem to find minimum cost SCC.

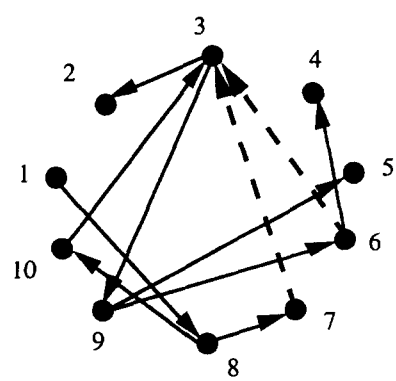



Fig. 1(b). Admissible arcs in the 1-arborescence solution.

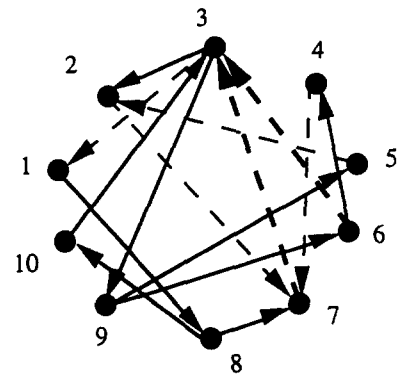



Fig. 1(c). Admissible arcs in 1-arborescence and in 1-anti arborescence solution.

indicated by "thin" dashed lines. In the admissible graph of Fig. 1(c), there is a path between every pair of nodes and hence the graph forms a SCC.

*Step 4.* In the resulting strongly connected graph $G_2$, it is possible that some nodes have odd degrees. To make the degrees of each node even, we add additional arcs between the odd degree nodes. This step is accomplished by the algorithm MATCH which solves a minsum matching problem on odd degree nodes making the degree of all nodes even. To solve the minsum matching problem we use the algorithm given in Ref. [31].

*Step 5.* Though the resulting graph $G_2$ now has all even degree nodes, there may yet exist nodes whose in-degrees are not equal to their out-degrees. We rectify this condition using Algorithm INOUT given by Edmonds and Johnson [12] which makes the in-degree of each node equal to its out-degree by orienting and duplicating the undirected edges.

*Steps 6 and 7.* Now the graph $G_2$ has been made Eulerian and an Euler tour can be found in this graph. Algorithm EULER finds the Euler tour in the resulting graph. This Euler tour is a large tour (Giant Euler tour) which covers all the specified arcs and specified nodes. This tour can be considered as a large single vehicle route which ignores the capacity and time restrictions. However, this Giant tour can be followed to generate individual multi-vehicle routes. Algorithm PRTOUR breaks the Giant Euler tour into smaller tours to satisfy the capacity and shift time constraints, thus forming feasible routes for each vehicle.

### *3.2. An Example of Procedure ROUTE1*

The algorithm ROUTE1 is explained using a small example. Figure 2 shows the initial network of streets and nodes. Arrows in Fig. 2 indicate arcs that are one way streets. The "black dots" represent specified nodes. A shortest path algorithm is applied to the network in Fig. 2 to find the shortest path between every pair of nodes. For the sake of clarity we do not show various parameters such as distance, weights etc. associated with each segment and node in Fig. 2.

Algorithm EXTRACT is applied to the graph in Fig. 2. The output of the algorithm EXTRACT is shown in Fig. 3. The arcs and nodes in Fig. 3 are the specified streets and nodes that need to be visited. However, this condensed network is not connected. To connect the network, algorithm MINSPA is applied to the graph. The output of the algorithm is a connected network. The edges (15, 17), (17, 19), (19, 21), (21, 23), (23, 25) and arc (29, 30) are added to make the graph connected as shown in Fig. 4.

To form tours traversing the arcs in the resulting connected network, there must be a path to go in and out of every node. Since, there may be nodes which violate this requirement, algorithm SCC is applied to the resulting graph. This algorithm first identifies the nodes which violate the requirement and then adds arc to make them feasible. The augmenting arcs are determined by solving a minimum weight strongly connected component problem. The output of the algorithm SCC adds arc (30, 58) to the graph and is shown by a bold dark line in Fig. 4. In this resulting graph there is a path between every pair of nodes.

The strongly connected graph contains a number of odd degree nodes. For example, nodes 3, 4, 11, 12, 25 are odd degree nodes. To have a feasible tour through all the arcs in graph, the degree

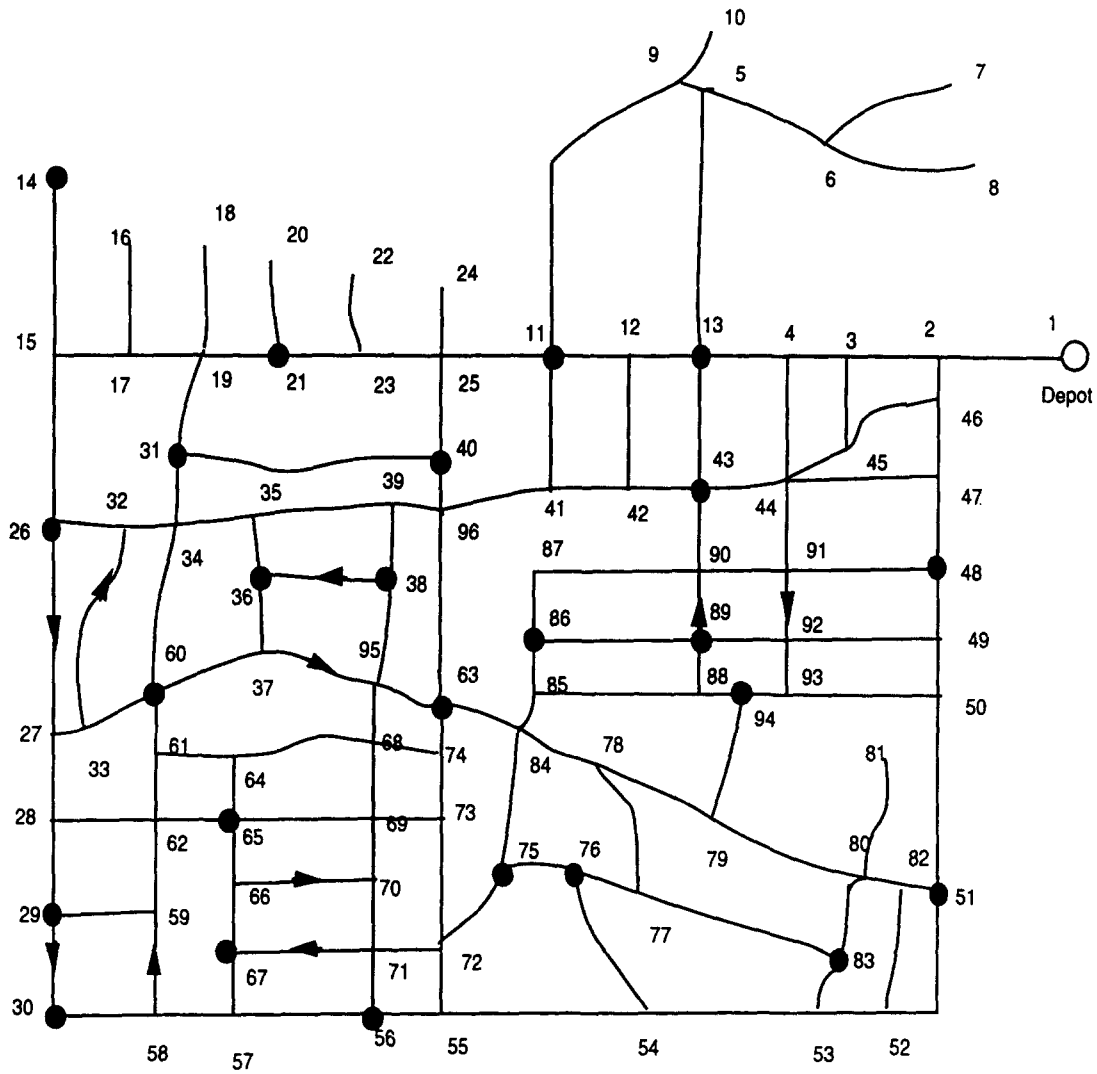



Fig. 2. Initial network.

of every node must be even. Therefore, all the odd degree nodes are found and a min-cost matching is done using algorithm MATCH to make the degree of all odd degree nodes even and the resulting graph is shown in Fig. 4 and the arcs added are shown using curved lines.

The nodes of the resulting graph have even degrees. To form an Euler tour, one need to orient the undirected arcs. This orientation must be performed such that each node has equal in and out degrees. Otherwise it may result in an infeasible Euler Tour. Therefore, the Algorithm INOUT is applied to the resulting graph to make the in-degree of every node equal to its out-degree and the output is shown in Fig. 4 with arrows on the directed arcs. Algorithm EULER is applied next to the resulting graph to form the giant Euler Tour. The Giant Euler Tour is then broken into smaller tours subject to capacity and time constraints. The individual tours are not listed due to brevity.

## 4. LOWER BOUNDS AND COMPUTATIONAL RESULTS

The performance of the developed heuristic procedures were evaluated by comparing the solutions with the lower bounds to the CCPP on some randomly generated test problems. The lower bounds we use are the Matching Lower Bound (MLB) proposed by Golden and Wong [24] and the Node Scanning Lower Bound proposed by Golden *et al.* [32)] for CCPP with some modifications. In the computation of lower bounds, the above studies consider only the capacity constraints and the restrictions due to available shift time are ignored. We then modified these procedures to explicitly

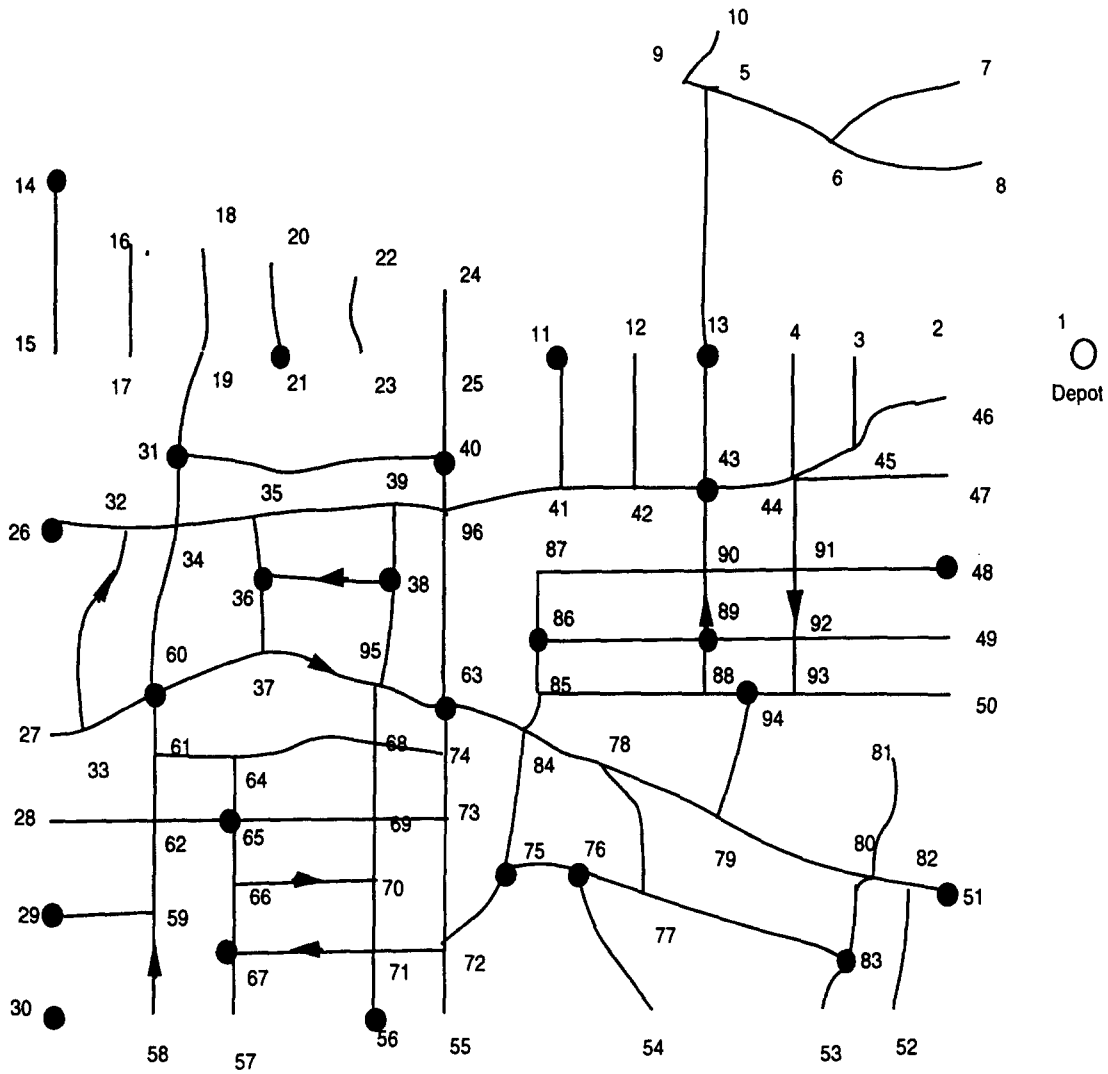



Fig. 3. Disconnected condensed network.

incorporate *both* the capacity and shift time restrictions in our lower bound computations. The incorporation of time restrictions in the lower bound computations results in stronger lower bounds when the travel and pick up times are significant. These modified bounding procedures are described as follows.

In the MLB procedure, the degree of the depot node must be increased from its original value $I$ to $2K_0$, where $K_0$ denotes a lower bound on the minimum number of cycles based at the depot in the CCPP solution and is computed as

$$K_{01} = \left\lceil \frac{\sum_i \sum_j q_{ij}}{W} \right\rceil,$$

$W$ = maximum vehicle capacity,

$$K_{02} = \left\lceil \frac{\sum_i s_i + \sum_j (C(i, j)/\text{Avg. speed} + t_{ij})}{T} \right\rceil,$$

$$K_0 = \max(K_{01}, K_{02}).$$

We assume that $J = 2K_0 - I$ is a positive even integer. Let $t_{ij}$ be the time required for pickup for the arc $(i, j)$. Let $S$ be the set of all nodes of odd degree in $G_0$. Define two new sets of nodes

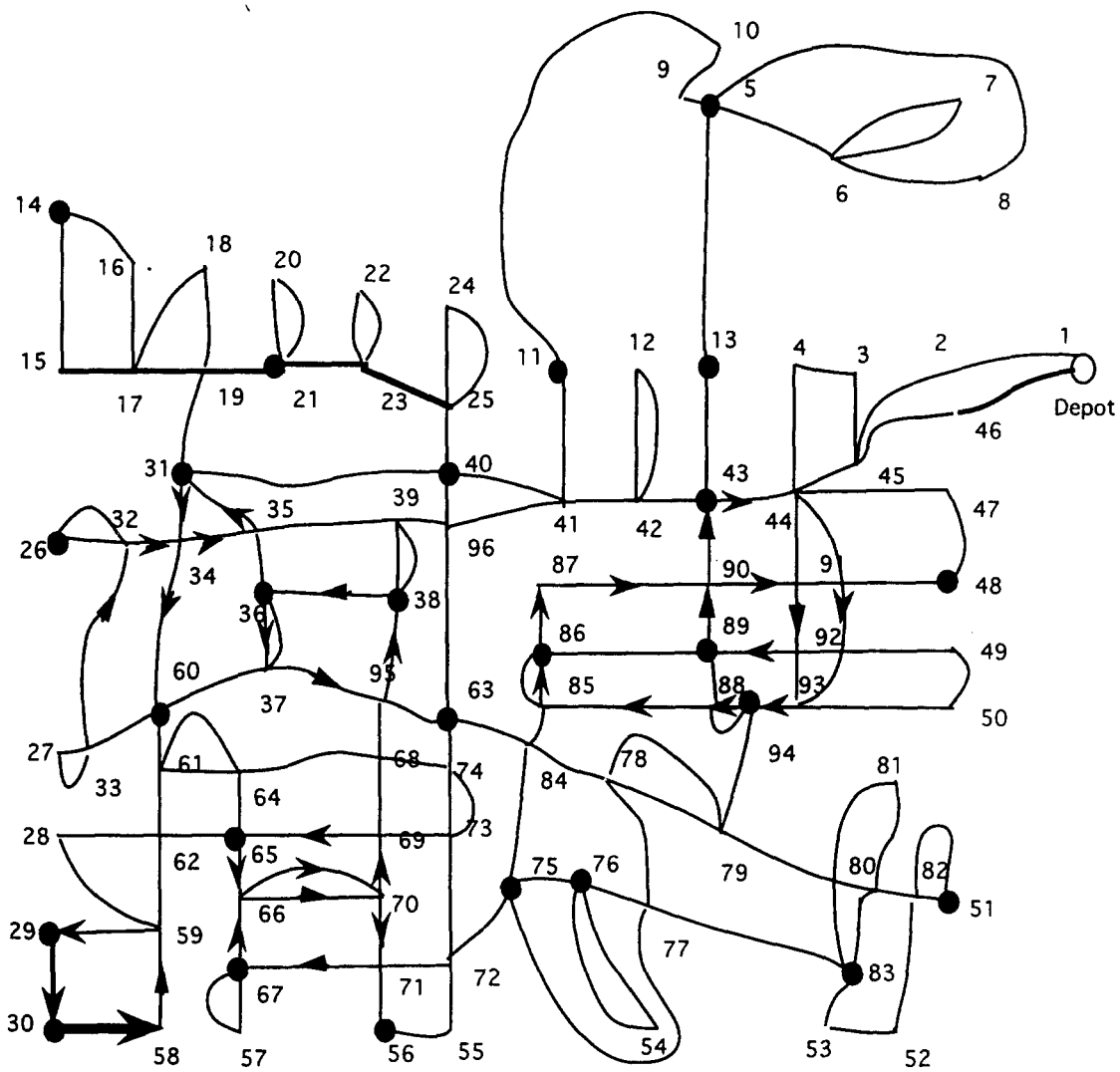



Fig. 4. Final Eulerian graph having equal in and out degrees.

$A=\{a_1,\ldots,a_j\}$ and $B=\{b_1,\ldots,b_j\}$ and the following set of edges on the node set $S\cup A\cup B$:

(1) Any pair of nodes $(i, j)$ in $S$ are joined by an edge of cost $C(i, j)$.
(2) Each node $i\in S$ is joined to all nodes in $A$ by edges of cost $C(1, i)$.
(3) The nodes $a_l$ and $b_l$ are joined by an edge with cost $C_0$ (for $l=1,\ldots,J$). $(C_0=\min\{C(1, i)\})$.
(4) Any pair of nodes in $B$ is connected by an edge of zero cost.

The network with node set $S\cup A\cup B$ and the set of edges defined above are called the matching network and are denoted by $F$. Let $M(F)$ represent the value of the minimum cost 1-matching on $F$. Golden and Wong [24] proved that

$\text{MLB}=|G_0|+M(F)$ is a lower bound for CCPP, where $|G_0|$ represent the sum of the lengths of all arcs that must be visited.

The second lower bound used for the CCPP is the Node Scanning Lower Bound (NSLB). This bound is easy to compute and may outperform the MLB in certain cases.

As with the matching lower bound, the node scanning bound increases the degree of the depot node by $J$. In the case of NSLB, however this is accomplished by adding minimum cost paths from the nodes to depot in the order of the path lengths, with the shortest path added first. Formally,

the NSLB procedure may be stated as:

(1) Renumber the nodes according to the "length" of the least cost path to the depot (node 1), so that $C(1,2) \leqslant C(1,3) \leqslant \ldots \leqslant C(1,n)$
(2) *Let* $l = \min\{k \mid \sum_{i=2}^{k} d(i) \geqslant J\}$, where $d(i)$ is the degree of node $i$.
Reset $d(l) = J - \sum_{i=2}^{l-1} d(i)$.
(3) Compute $\text{NSLB} = |G_0| + \sum_{i=2}^{l} C(1,i)d(i)$ as a lower bound for the CCPP.

### *4.1. Computational Results*

The developed algorithms, which are outlined in Appendix I, were coded in FORTRAN 77. The size of the problem that can be solved using these algorithms depends on the following factors:

1. Number of nodes (junctions) of the initial network.
2. Number of edges and arcs (streets) of the initial network.
3. Number of nodes, edges and arcs to be visited.
4. Number of directed arcs.

The test problems were generated with respect to a curb-side waste-collection problem arising in residential areas. In this problem, waste is to be collected from the curb-sides and convenience stores. These curb-sides and stores form specified set of arcs, edges and nodes of the travel network and the vehicles must visit these. The problem sets were generated randomly using Prufer Code [33] to evaluate the effect of the number of directed arcs and the number of arcs to be visited on system performance. Test problems with 50 nodes and 100 arcs are generated in two sets. In the first set, the number of directed arcs in the problem is varied as 0, 20, 40 and 60% of the total number of arcs. The recyclables are to be collected from 90% of the arcs in the network in each case. For the second set of problems, 20% of the arcs were kept directed and the percentage of arcs to be visited are varied as 100, 80, 60 and 40%. The computational results for the first set are presented in Tables 1–4 and results for the second set are given in Tables 5–8. In these tables, LB1 is the Matching based lower bound and LB2 is the Node Scanning lower bound.

In each case, the parameters of the five test problems were generated using seeds 1357, 7382,

Table 1. Comparison of solution with lower bounds for 0% of arcs directed

| Problem No. | Solution | CPU s | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 139.9 | 40.57 | 90.05 | 87 | 1.55 | 1.61 |
| 2 | 139.55 | 37.80 | 80.15 | 86.15 | 1.74 | 1.62 |
| 3 | 145.25 | 38.9 | 89.45 | 88.25 | 1.62 | 1.65 |
| 4 | 133.8 | 38.67 | 82.95 | 78.2 | 1.61 | 1.71 |
| 5 | 126.2 | 37.74 | 72.4 | 77.9 | 1.74 | 1.62 |

Table 2. Comparison of solution with lower bounds for 20% of arcs directed

| Problem No. | Solution | CPU | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 138.45 | 40.88 | 90.05 | 87 | 1.54 | 1.59 |
| 2 | 144.1 | 38.77 | 80.15 | 86.15 | 1.80 | 1.67 |
| 3 | 143.85 | 39.59 | 89.45 | 89.05 | 1.61 | 1.62 |
| 4 | 142.35 | 40.32 | 82.95 | 80.2 | 1.72 | 1.77 |
| 5 | 143.2 | 43.71 | 73.5 | 79.55 | 1.95 | 1.80 |

Table 3. Comparison of solution with lower bounds for 40% of arcs directed

| Problem No. | Solution | CPU | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 172.65 | 38.29 | 90.5 | 88.95 | 1.91 | 1.94 |
| 2 | 170.01 | 50.57 | 82 | 87.8 | 2.07 | 1.94 |
| 3 | 164.05 | 41.48 | 89.85 | 89.7 | 1.83 | 1.83 |
| 4 | 169.35 | 46.12 | 83.75 | 79.8 | 2.02 | 2.12 |
| 5 | 151.85 | 42.45 | 74 | 80.5 | 2.05 | 1.89 |

Table 4. Comparison of solution with lower bounds for 60% of arcs directed

| Problem No. | Solution | CPU | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 187.4 | 47.61 | 90.6 | 90.65 | 2.07 | 2.07 |
| 2 | 201.9 | 55.08 | 82.6 | 90.65 | 2.44 | 2.23 |
| 3 | 201.65 | 52.15 | 90.5 | 92.35 | 2.23 | 2.18 |
| 4 | 218.4 | 57.03 | 106.2 | 91.7 | 2.06 | 2.38 |
| 5 | 184.65 | 54.21 | 85.2 | 85.75 | 2.17 | 2.15 |

Table 5. Comparison of solution with lower bounds for 100% of arcs to be visited

| Problem No. | Solution | CPU | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 168 | 39.17 | 92.25 | 101.65 | 1.82 | 1.65 |
| 2 | 166.5 | 38.44 | 91.05 | 98.35 | 1.83 | 1.69 |
| 3 | 155.05 | 39.37 | 92.45 | 94.75 | 1.68 | 1.64 |
| 4 | 153.85 | 37.13 | 86.65 | 90.3 | 1.77 | 1.70 |
| 5 | 148.2 | 37.15 | 82.25 | 91.7 | 1.80 | 1.62 |

Table 6. Comparison of solution with lower bounds for 80% of arcs to be visited

| Problem No. | Solution | CPU | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 122.95 | 33.26 | 77.6 | 74.15 | 1.58 | 1.66 |
| 2 | 134 | 34.21 | 81.6 | 79.5 | 1.64 | 1.69 |
| 3 | 148.25 | 40.03 | 84.6 | 80.1 | 1.75 | 1.85 |
| 4 | 137.01 | 38.61 | 74.55 | 69.65 | 1.84 | 1.97 |
| 5 | 144.35 | 40.2 | 68.8 | 71.65 | 2.10 | 2.01 |

Table 7. Comparison of solution with lower bounds for 60% of arcs to be visited

| Problem No. | Solution | CPU | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 96.55 | 36.21 | 58.05 | 56.1 | 1.66 | 1.72 |
| 2 | 112.3 | 45.48 | 64.7 | 61.05 | 1.74 | 1.84 |
| 3 | 119.45 | 39.30 | 70.2 | 66.45 | 1.70 | 1.80 |
| 4 | 124.8 | 38.30 | 57.6 | 53.8 | 2.17 | 2.32 |
| 5 | 127.4 | 45.84 | 60.55 | 60.95 | 2.10 | 2.09 |

Table 8. Comparison of solution with lower bounds for 40% of arcs to be visited

| Problem No. | Solution | CPU | LB1 | LB2 | Percentage of LB1 | Percentage of LB2 |
|---|---|---|---|---|---|---|
| 1 | 95.85 | 58.34 | 53.35 | 46.45 | 1.80 | 2.06 |
| 2 | 98.85 | 52.77 | 64.15 | 54.4 | 1.54 | 1.82 |
| 3 | 95.7 | 64.68 | 55.75 | 46.45 | 1.72 | 2.06 |
| 4 | 98.4 | 53.23 | 41.2 | 34.75 | 2.39 | 2.83 |
| 5 | 85.1 | 68.50 | 51.8 | 41.7 | 1.64 | 2.04 |

5361 and 9136, respectively. Fifty per cent of the nodes in the graph are assumed to be the specified nodes. The parameters are generated for all test problems using discrete uniform distribution as follows. The number of bags to be picked up from the specified nodes is generated in the range [1, 15]. The pick up time is generated in the range [15, 30]. The number of bags to be collected from the street is generated in the range [1, 15]. The length of the streets is generated in the range [0.25, 1] miles. The vehicle capacity is 30 bags and its speed is taken as a constant 25 miles per hour. It is assumed that a route can be at most 4 hours long.

From the computational results as shown in Table 1, when 0% of the arcs are directed (undirected network) then the algorithm finds a solution whose value is on the average 1.65 times more than the value of the lower bound in each case. Tables 2–4 show the results when 20, 40 and 60% of

the arcs in the network are directed respectively. As the percentage of arcs directed in the network increases, the ratio of the solution to the lower bound increases. This may be attributed to a weak lower bound. The lower bound proposed by Golden and Wong is for an undirected network. For test problems with 60% of the directed arcs, this bound results in a weaker lower bound with a ratio of solution to lower bound as 2.2. The CPU time increases with number of arcs directed because as the percentage of directed arcs increases, the number of components in the graph increases. Consequently, more time is spent in making the graph strongly connected. Hence, for 0% of arcs directed the CPU time is the least. The solution value also increases as the number of directed arcs increases because the distance to reach some of the nodes or arcs from the depot increases as the arcs are directed.

The solution value decreases as the percentage of the arcs to be visited are decreased because relatively less travel time is required as shown in Tables 5–8. The CPU time first decreases with the decrease in the number of arcs to be visited due to smaller problem size. However, as the number of arcs to be visited is decreased further, the number of disconnected components in the resulting condensed graph increases. Consequently, more CPU time is required to make the graph strongly connected.

## 5. SUMMARY

In this study, the problem of routing of vehicles over the specified segments of a mixed network is addressed. The problem is modeled as a CGRP. Since this problem is NP-hard, a heuristic procedure is developed to solve this problem. The problem of finding minimum weight strongly connected components of a graph arises as a sub-problem. This is a NP-hard problem for its general case. We present a heuristic procedure to solve it.

The developed software is packaged as a micro-computer based interactive optimization system. The software is user friendly, detects input data errors, provides help and is interactive. The user can also evaluate his/her own routes against those generated by the software to compare the alternate routing strategies. The overall system can be used for planning, cost estimation or for daily operations.

*Acknowledgements*—The authors wish to thank Plastics Recycling Foundation, Inc., Washington, DC for providing financial support for this study through the Center for Plastics Recycling Research, Building 3529, Busch Campus, Rutgers, Piscataway, NJ 08855. The authors also thank two anonymous referees and Dr Jeff Tew for their helpful comments on an earlier version of this paper.

## REFERENCES

1. E. J. Beltrami and L. D. Bodin, Networks and vehicle routing for municipal waste collection. *Networks* **4**, 65–94 (1974).
2. L. Bodin and J. Kursh, A detailed description of computer systems for the routing and scheduling of street sweepers. *Computer ops Res.* **6**, 181–198 (1979).
3. L. Bodin, G. Fagin, R. Webno and J. Green Beeg, The design of computerized sanitation vehicle routine and scheduling system for the town of Oyster Bay. *Computers ops Res.* **16**, 45–54 (1989).
4. H. Stern and M. Dorr, Routing electric meter readers. *Computers ops Res.* **6**, 209–223 (1979).
5. J. A. Ferland, L. Chapleau, G. Lapalme and J.-M. Rousseau, A parallel insert method for the capacitated arc routing problem. *OR Letts* **3** (2) (1984).
6. L. Bodin, B. Golden, A. Assad and M. Ball, Routing and scheduling of vehicles and crews, the state of the art. *Computers ops Res.* **10**, 2 (1983).
7. L. Bodin, The state of art in the routing and scheduling of crews. A Report Submitted to the U.S. Department of Transportation, Urban Mass Transportation Administration, Washington, DC (1981).
8. L. Bodin and B. Golden, Classification in vehicle routing and scheduling. *Networks* **11**, 97–108 (1981).
9. L. Bodin, Twenty years of routing and scheduling. *Ops Res.* **38** (4) July–August (1990).
10. L. Bodin and Edward A. Wasil, Computerized vehicle routing in the soft drink industry. *Ops Res.* **35** (1) January–February (1987).
11. L. Bodin, B. L. Golden and T. Goodwin, Microcomputer-based vehicle routing and scheduling software. *Computers ops Res.* **13**, 277–285 (1986).
12. J. Edmonds and E. Johnson, Matching, Euler Tours and the Chinese postman. *Math. Prog.* **5**, 88–124 (1973).
13. G. N. Frederickson, M. S. Hecht and C. E. Kim, Approximation algorithms for some routing problems. *SIAM J. Comput.* **7** (2) May (1978).
14. G. N. Frederickson, Approximation algorithms for some postman problems. *J. Ass. Comput. Mach.* **26**, 538–554 (1979).
15. E. Minieka, The Chinese postman problem for mixed networks. *Mgmt Sci.* **25** (7), July (1979).
16. B. L. Golden, J. J. De Armon and E. K. Baker, Computational experiments with algorithms for a class of routing problems. *Computers ops Res.* **10**, 47–59 (1983).
17. B. L. Golden, T. Magnanti and H. Neguyen, Implementing vehicle routing algorithms. *Networks* **7**, 113–148 (1977).

18. W. L. Pearn, Augment insert algorithms for the capacitated arc routing problem. *Computers ops Res.* **18**, 189–198 (1991).
19. N. Christofides, The optimal traversal of a graph. *Omega*, 719–732 (1973).
20. W. L. Pearn, Approximate solutions to the capacitated arc routing problem. *Computers ops Res.* **16**, 589–600 (1989).
21. C. Orloff, A fundamental problem in vehicle routing. *Networks* **4**, 147–162 (1974).
22. C. Orloff, On general routing problems: comments. *Networks* **6**, 281–284 (1976).
23. J. Lenstra and A. Rinnooy Kan, On general routing problems. *Networks* **6**, 273–380 (1976).
24. B. L. Golden and R. T. Wong, Capacitated arc routing problems. *Networks* **7**, 113–148 (1977).
25. D. T. Phillips and A. Garcia-Diaz, *Fundamentals of Network Analysis*. Prentice-Hall, Englewood Cliffs, NJ (1981).
26. R. Tarjan, Depth first search and linear graph algorithms. *SIAM Jl Comput.* **1** (2) June (1972).
27. K. P. Eswaran and R. Tarjan. Augmentation problems. *SIAM Jl Comput.* **5** (4), July–August (1987).
28. M. Fischetti and P. Toth, An additive bounding procedure for the asymmetric traveling salesman problem. Technical Report DEIS, University of Bologna (1987).
29. M. Fischetti and P. Toth, An additive bounding procedure for combinatorial optimization problem. *Ops. Res.* **37**, 319–328 (1989).
30. M. Fischetti and P. Toth, An efficient algorithm for the min-sum arborescence problem. Technical Report, DIES, University of Bologna (1988).
31. E. R. Burkard and U. Derigs, Assignment and matching problems: solution methods with FORTRAN-PROGRAMS. *Lecture Notes in Economics and Mathematical Systems*. Springer, New York (1980).
32. B. L. Golden, Arjang A. Assad and Wen-Lea Pearn, The capacitated Chinese postman problem: lower bounds and solvable cases. *Am. J. Mathl Mgmt Sci.* **7** (1/2) (1987).
33. C. M. Murphy and M. Shing Hung, Network generation using prufer code. *Computers ops Res.* **13**, 693–705 (1986).

## APPENDIX I

*Algorithm ROUTE1*

*Input*

A mixed graph $G_0=(N_0, R_0)$ and distance $C(i, j)$ for each arc $(i, j) \in R_0$, $t_{ij}$ pickup time for arc $(i, j)$, $W_k$ capacity of the vehicle $k$, $T$ shift time, $q_{ij}$ capacity to be collected in street $(i, j)$, $p_i$ capacity to be collected at node $i$, $s_i$ pick up time at node $i$, and the speed of the vehicle, $K$ the total number of vehicle.

*Output*

Tours that cover all the edges, arcs and nodes that need to be visited and satisfy the capacity and time constraints.

Begin
Step 1. Call EXTRACT.
Step 2. Call MINSPA.
Step 3. Call SCC.
Step 4. Call MATCH.
Step 5. Call INOUT.
Step 6. Call EULER.
Step 7. Call PRTOUR.
End.

*Algorithm EXTRACT*

*Input*

A mixed graph $G_0=(N_0, R_0)$.

*Output*

Disconnected components, distance between them and graph $G_1=(V_1, R_1)$.

Begin
Step 1. Identify the edges and arcs that needs to be visited and set them equal to $R_1$.
Step 2. Identify the nodes to be visited and set them equal to $V_1$.
Step 3. Find if the graph $G_1=(V_1, R_1)$ is connected (ignoring the arc direction).
Step 4. If not connected find out the minimum distance between the disconnected components.
End.

*Algorithm MINSPA*

*Input*

Composite nodes (Disconnected components) and the distance between them.

*Output*

The Minimum Spanning Tree connecting the different components.

Begin
Step 1. Construct graph $H$ from $G_1$.
Step 1. Find the minimum spanning tree spanning the composite nodes.
End.

*Algorithm SCC*

*Input*

A connected graph $G_2=(V_2, R_2)$.

*Output*

Strongly connected Graph.

Begin

Step 1. Find the Strongly Connected Components (without ignoring the arc direction) as in Ref. [26].

Step 2. If there is more than one component, form graph Ω. Find Min Sum 1-Arborescence and 1-Anti Arborescence as in Refs [28, 30].

Step 3. Add the arcs in the solution vector to $G_2$ to form a strongly connected graph.

End.

*Algorithm MATCH*

*Input*

A strongly connected graph $G_2=(V_2, R_2)$.

*Output*

A mixed Graph $G_2$ in which the degree of each vertex is even.

Begin

Step 1. Identify the vertices of odd degree.

Step 2. Perform a minimum-cost matching of the odd vertices using the shortest path distances as in Ref. [31].

Step 3. Add arcs in the solution of the matching to graph $G_2$.

End.

*Algorithm INOUT*

*Input*

A graph $G_2=(V_2, R_2)$ with all even degree nodes.

*Output*

A Graph $G_2$ in which in-degree equals out-degree for every vertex and contains all the arcs and nodes that needs to be visited

Begin

Step 1. Solve a network minimum cost flow problem using Out-of-Kilter Algorithm [25] and make in-degree of each node equal to it's out-degree as in Ref. [12].

End.

*Algorithm EULER*

*Input*

Output from the Algorithm INOUT-graph $G_2$.

*Output*

Giant Euler Tour

Begin

Step 1. Find the Euler tour in the Graph $G_2$.

End.

*Algorithm PRTOUR*

*Input*

Giant Euler Tour

*Output*

Smaller tours that cover all the edges and nodes that need to be visited and tours satisfy the capacity and time constraints of the vehicle.

Begin

Step 1. Follow the Euler Tour, break the Tour whenever the capacity or time constraints are violated and return to the depot. Start the tour from the previous break point.

End.